Person B guide: transfer learning, augmentation, rejection

Written to you, Person B. Every part refers to the real file and function. Read with the code open.

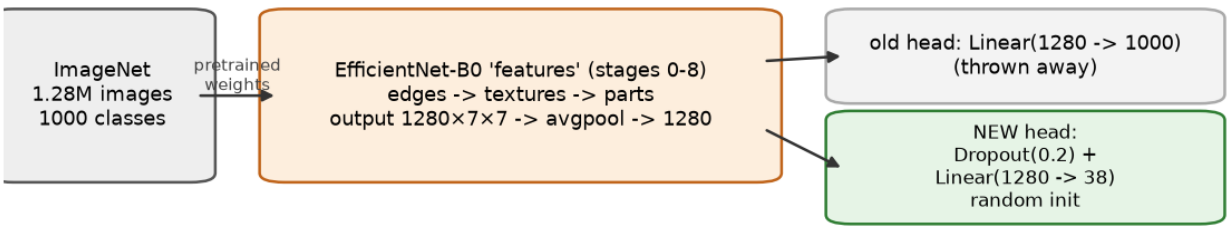
0. Your part in one paragraph

You built the **transfer-learning model**: EfficientNet-B0 pretrained on ImageNet and fine-tuned on our leaves. You evaluate it like the baseline, run the **augmentation experiments** that try to reduce the gap between lab photos (PlantVillage) and field photos (PlantDoc), and you add a **confidence threshold** so the app says 'not sure' instead of guessing.

File: function	What it does	Why
models.py: build_efficientnet, get_model	loads EfficientNet-B0 with ImageNet weights, new 38-class head	transfer learning
train.py: run_experiment, fit	the same training loop as the baseline (AdamW, cosine LR, best val checkpoint)	fair comparison
data.py: get_transforms('standard' / 'strong')	the augmentation pipelines	the augmentation experiment
evaluate.py: evaluate_experiment, results_table, plot_domain_shift	PV test, PV shared classes, PlantDoc; domain gap	measures the lab-to-field drop
evaluate.py: rejection_analysis, coverage_and_accuracy	chooses the threshold on val, reports coverage and accuracy	honest 'not sure' answers
app.py: find_threshold, build_app	uses the threshold in the Gradio demo	deployment behaviour

1. What transfer learning is and why it works

A CNN trained on ImageNet (1.28 million photos, 1000 classes) has learned a hierarchy of features. Early layers detect edges and colour blobs, middle layers textures and simple shapes, late layers object parts. Most of this is useful for any photo, including leaves with spots. **Transfer learning** re-uses these weights as the starting point instead of random numbers. Our ~7.4k training images are then only needed to adapt the features to leaf diseases, not to learn vision from zero. Typical result: higher accuracy, in fewer epochs, with less overfitting.



OUR runs (fine-tuning): every layer is trained, lr 0.0005, 4,056,226 trainable parameters

feature extraction (option freeze_backbone=True, NOT used in our runs): only the head is trained, 48,678 parameters

What build_efficientnet (src/models.py) does.

2. EfficientNet-B0 at a high level

EfficientNet-B0 (Tan & Le, 2019) is a CNN built from **MBConv** blocks: a cheap 1×1 conv expands the channels, a **depthwise** 3×3 or 5×5 conv filters each channel separately (much cheaper than a normal conv), a **squeeze-and-excitation** step re-weights channels by importance, and a 1×1 conv shrinks the channels again, with a skip connection. Activation is SiLU (x·sigmoid(x)). 'B0' is the smallest member of the family; bigger versions scale depth, width and resolution together ('compound scaling').

Real output shapes and parameter counts of the torchvision model with our 38-class head (forward pass of one 224×224 image):

Part	Type	Output shape	Parameters
features[0]	Conv2dNormActivation	32×112×112	928
features[1]	Sequential	16×112×112	1,448
features[2]	Sequential	24×56×56	16,714
features[3]	Sequential	40×28×28	46,640
features[4]	Sequential	80×14×14	242,930
features[5]	Sequential	112×14×14	543,148
features[6]	Sequential	192×7×7	2,026,348
features[7]	Sequential	320×7×7	717,232
features[8]	Conv2dNormActivation	1280×7×7	412,160
avgpool	AdaptiveAvgPool2d	1280×1×1	0
classifier[0]	Dropout	1280	0
classifier[1]	Linear	38	48,678

Total parameters: **4,056,226**, of which trainable in our runs: **4,056,226** (all of them). With `freeze_backbone=True` only **48,678** would be trainable (the new head).

```
weights = models.EfficientNet_B0_Weights.IMAGENET1K_V1 if pretrained else None
model = models.efficientnet_b0(weights=weights)
if freeze_backbone:
    for p in model.features.parameters():
        p.requires_grad = False
in_features = model.classifier[1].in_features  # 1280
model.classifier[1] = nn.Linear(in_features, num_classes)  # new head, randomly initialised
return model
```

Source: src/models.py: build_efficientnet (line 51).

3. What we froze or unfroze, and why

	Feature extraction	Fine-tuning (what we ran)
Pretrained layers	frozen (requires_grad=False)	updated, with a small learning rate
Trainable parameters	48,678 (head only)	4,056,226 (all)
Pros	very fast, little overfitting with tiny data	features adapt to leaves and diseases; usually more accurate when the data differ from ImageNet
In our code	`build_efficientnet(freeze_backbone=True)`, never called by the notebook	`get_model('effnet', 38)` -> freeze_backbone=False

Why we fine-tune everything: leaf disease spots look nothing like ImageNet's dogs and cars, we have several thousand images (enough to adapt the backbone a little), and the small

learning rate protects the pretrained features. Be honest: we did not run the frozen variant, so we cannot show numbers that fine-tuning is better here.

4. Training setup and learning-rate schedule

Setting	Value	Where in the code	Why
Loss	CrossEntropyLoss	train.py: fit	standard loss for picking one of many classes; includes the softmax, so the model outputs raw logits
Optimizer	AdamW, weight decay 0.0001	train.py: fit	Adam adapts the step size per weight; the 'W' version applies weight decay correctly (regularisation)
Learning rate	0.0005	notebook settings: EFFNET_LR	scratch CNN can take bigger steps; a pretrained net needs smaller steps so it doesn't destroy ImageNet features
LR schedule	CosineAnnealingLR, T_max=6, stepped per epoch	train.py: fit	large steps first, small careful steps at the end
Batch size	64	notebook settings: BATCH_SIZE	fits easily in T4 memory; 64 images give a stable gradient estimate
Epochs	6	notebook settings: EFFNET_EPOCHS	keeps all runs within ~40 min on a T4; a pretrained model converges fast
Model selection	best validation accuracy	train.py: fit	picks the epoch that generalises best without touching the test set
Seed	42	utils.py: set_seed (called in run_experiment)	same split, init and shuffling every run
Mixed precision	on GPU only (autocast + GradScaler)	train.py: train_one_epoch	about 2× faster on a T4 with the same accuracy
Input	3×224×224, ImageNet mean/std	data.py: get_transforms	same input for both models, so the comparison is fair

The real learning rate in each epoch (same scheduler, recomputed here):

Epoch	1	2	3	4	5	6	7	8	9	10	11	12
SimpleCNN	1.0e-03	9.8e-04	9.3e-04	8.5e-04	7.5e-04	6.3e-04	5.0e-04	3.7e-04	2.5e-04	1.5e-04	6.7e-05	1.7e-05
EfficientNet	5.0e-04	4.7e-04	3.8e-04	2.5e-04	1.3e-04	3.3e-05						

Cosine annealing: lr_e = lr_max · (1 + cos(π·e / T_max)) / 2 for epoch e = 0 … T_max−1. Big steps while the head is still random, then smaller and smaller steps to settle into a good minimum.

5. Evaluation of the transfer model

The same functions as for the baseline (`evaluate_experiment`): accuracy, macro precision/recall/F1, a row-normalised confusion matrix (diagonal = recall), on three test sets: PlantVillage test (38 classes), PlantVillage test restricted to the 28 shared classes, and PlantDoc (28 classes).

Results are not available yet

• The numbers in `results/` are from a CPU smoke test (1 epoch, 2 images per class), so they are shown as [pending] here. After the Colab run: copy `results/` into the project and run `python scripts/build_viva_pdfs.py` again. All tables and figures fill in automatically.

Experiment	PV test acc (38)	PV macro-F1	PV acc (28 shared)	PlantDoc acc (28)	PlantDoc macro-F1	Domain gap
cnn_standard	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]
effnet_standard	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]
cnn_strong	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]
effnet_strong	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]

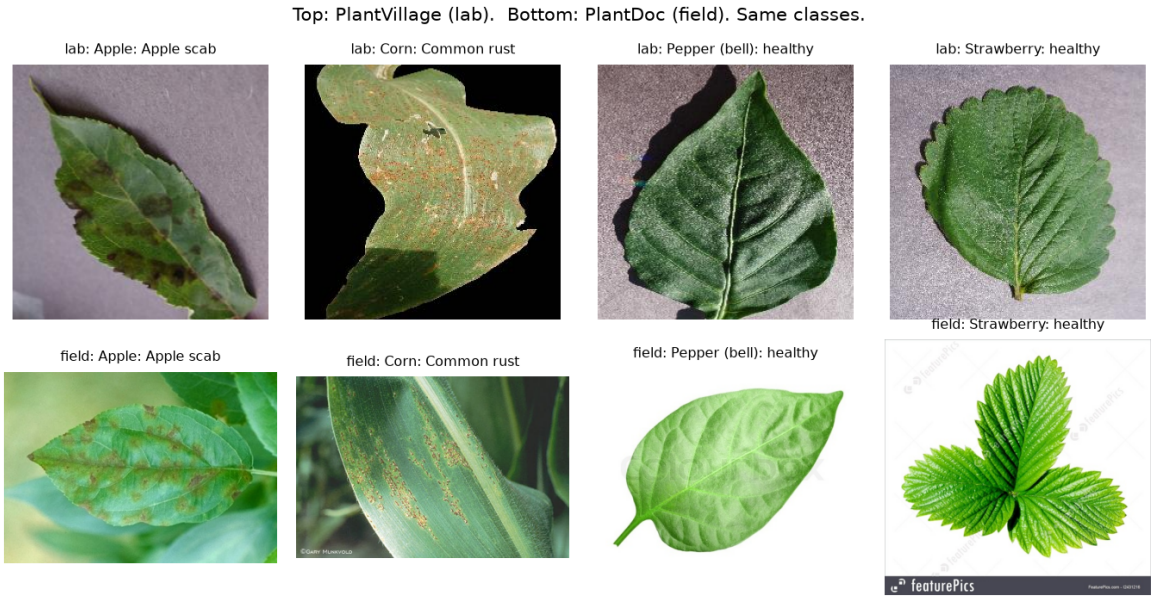
[After the Colab run, write 2-3 sentences: how much better is effnet_standard than cnn_standard on PlantVillage, and on PlantDoc? Which classes are hardest (per_class in `results/eval/effnet_standard_pv_test_metrics.json`)?]

[figure pending: `results/plots/history_effnet_standard.png` appears here after the Colab run]

[figure pending: `results/eval/effnet_standard_plantdoc_confusion.png` appears here after the Colab run]

6. Domain shift: lab to field

Domain shift = the test images come from a different distribution than the training images. Our model only saw PlantVillage: one centred leaf, plain background, controlled light. PlantDoc photos have soil, sky, hands, several leaves, shadows and blur. Even if the disease looks the same, the image statistics are different.



Same classes, lab (top) vs field (bottom).

- We compare on the **same 28 classes** so the comparison is fair.
- The model still picks among all 38 classes, as a real app would. Picking a class PlantDoc does not have simply counts as wrong (the 'other' column of the confusion matrix).
- **Domain gap = PV accuracy (shared classes) − PlantDoc accuracy.**
- PlantDoc photos are not square, and `Resize((224, 224))` squashes them; this distortion is part of the gap.

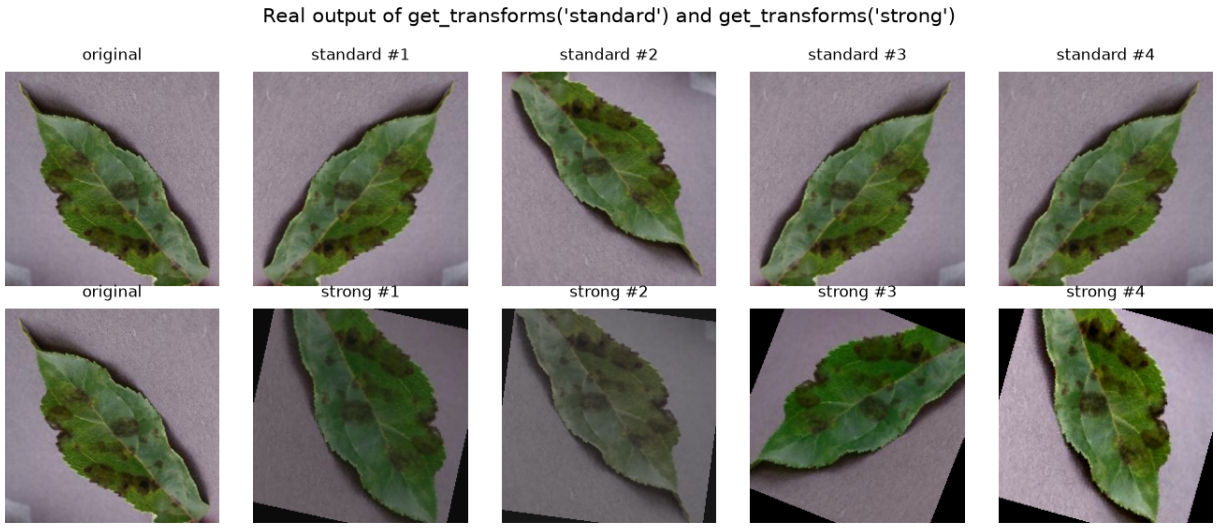
7. The augmentation experiments

Idea: if the training images already vary the way field photos vary, the model learns features that do not depend on those details. We retrain **both** models with 'strong' augmentation, with

everything else unchanged (same data, epochs, LR, seed), and compare before and after.

Transform (exact code)	What it does	Why it should help on field photos
RandomResizedCrop(size=(224, 224), scale=(0.5, 1.0), ratio=(0.75, 1.3333), interpolation=bilinear, antialias=True)	crops a random 50-100% area (aspect 3:4 to 4:3) and resizes it to 224	field photos are taken at different distances; the leaf is not always centred and full-size
RandomHorizontalFlip(p=0.5)	mirrors left-right with p=0.5	a leaf can point either way
RandomVerticalFlip(p=0.5)	mirrors top-bottom with p=0.5	photos are taken from any angle
RandomRotation(degrees=[-30.0, 30.0], interpolation=nearest, expand=False, fill=0)	rotates by a random angle in [-30°, 30°]; corners become black	hand-held photos are tilted
ColorJitter(brightness=(0.6, 1.4), contrast=(0.6, 1.4), saturation=(0.6, 1.4), hue=(-0.05, 0.05))	random brightness, contrast, saturation (±40%) and hue (±0.05)	sunlight, shade, different cameras and white balance
RandomApply(p=0.3 GaussianBlur(kernel_size=(9, 9), sigma=(0.1, 3.0)))	with p=0.3: Gaussian blur, kernel 9, sigma 0.1-3	out-of-focus or motion-blurred phone photos
ToTensor()	PIL image -> 3×224×224 tensor in [0, 1]	(needed, not augmentation)
Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])	ImageNet mean/std	(needed, not augmentation)

'Standard' augmentation (the 'before' runs) is only: `Resize(size=(224, 224), interpolation=bilinear, max_size=None, antialias=True)`; `RandomHorizontalFlip(p=0.5)`; `RandomVerticalFlip(p=0.5)`; `ToTensor()`; `Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`.



The same training image through get_transforms('standard') and ('strong').

Before / after

Results are not available yet

• The numbers in `results/` are from a CPU smoke test (1 epoch, 2 images per class), so they are shown as [pending] here. After the Colab run: copy `results/` into the project and run `python scripts/build_viva_pdfs.py` again. All tables and figures fill in automatically.

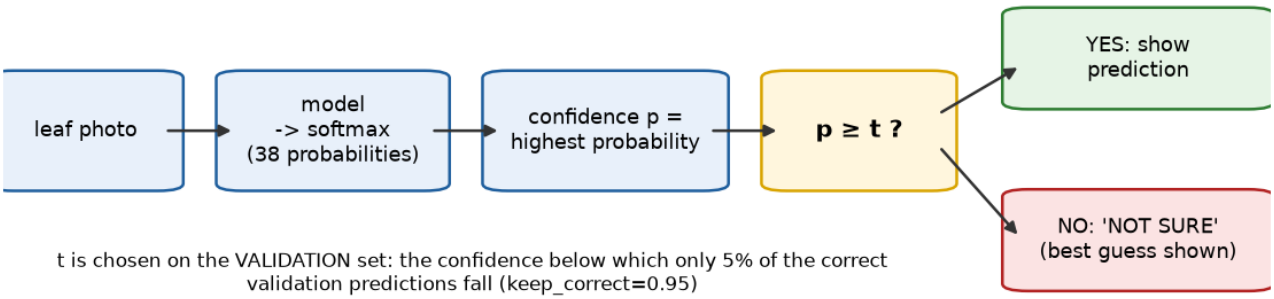
Model	Augmentation	PV acc (38)	PV acc (28)	PlantDoc acc (28)	Domain gap
SimpleCNN	standard (before)	[pending]	[pending]	[pending]	[pending]
SimpleCNN	strong (after)	[pending]	[pending]	[pending]	[pending]

Model	Augmentation	PV acc (38)	PV acc (28)	PlantDoc acc (28)	Domain gap
EfficientNet-B0	standard (before)	[pending]	[pending]	[pending]	[pending]
EfficientNet-B0	strong (after)	[pending]	[pending]	[pending]	[pending]

[figure pending: `results/plots/domain_shift.png` appears here after the Colab run]

How to read it: the gap shrinking from 'standard' to 'strong' means augmentation helped robustness. A small loss on PlantVillage with a gain on PlantDoc is a good trade. [Write your conclusion here.]

8. Confidence and the rejection threshold



How app.py decides between a prediction and 'NOT SURE'.

Any classifier always outputs some class, even for a blurry photo or a non-leaf. Rejection lets the system say 'I am not sure' when the confidence is low. **Confidence** = the softmax probability of the predicted class.

```
y_true, y_pred, probs = predict_dataset(model, pv_dataset(pv_dir, "val"), device,
                                        num_workers=num_workers)
conf_val, correct_val = probs.max(1), y_true == y_pred
if not correct_val.any():  # only possible in a tiny smoke test
    correct_val = np.ones_like(correct_val)
threshold = float(np.quantile(conf_val[correct_val], 1 - keep_correct))
```

- **How the threshold is chosen:** on the validation set only. Take the confidences of the validation images the model got right; the threshold is the value below which 5% of them fall (`keep_correct=0.95`). So on lab images about 95% of the correct answers survive.
- **Why the validation set:** choosing it on a test set would make the test numbers optimistic (leakage).
- **Coverage** = share of test images with confidence ≥ threshold (they get an answer). **Accuracy on answered** = accuracy on those images only (`coverage_and_accuracy`).
- **Trade-off:** a higher threshold means fewer answers (lower coverage) but more of them are right. The plot `results/rejection/<model>_coverage_accuracy.png` shows the whole curve for thresholds 0 to 0.99; the dot marks our threshold.
- Computed for effnet_standard and effnet_strong. The app reads `results/rejection/<checkpoint name>.json` (`find_threshold`); if the file is missing it uses 0.5, and `--threshold` overrides both.

Results are not available yet

• The numbers in `results/` are from a CPU smoke test (1 epoch, 2 images per class), so they are shown as [pending] here. After the Colab run: copy `results/` into the project and run `python scripts/build_viva_pdfs.py` again. All tables and figures fill in automatically.

Model	Threshold	PV test: answered	PV test: acc on answered	PV test: acc all	PlantDoc: answered	PlantDoc: acc on answered	PlantDoc: acc all
effnet_standard	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]
effnet_strong	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]	[pending]

[figure pending: `results/rejection/effnet_strong_coverage_accuracy.png` appears here after the Colab run]

What to expect and say: on PlantDoc the model should be less confident, so more images get rejected (lower coverage), and accuracy on the answered ones should be higher than accuracy on all of them. But softmax confidence is not calibrated: a field photo can be confidently wrong, so rejection reduces mistakes without removing them.

Things to double-check before the viva

These are places where the code is easy to misdescribe, or where something is limited or undocumented. Better to know them than to be surprised.

- The files now in `results/` come from a 1-epoch CPU smoke test on 2 images per class. They are NOT results. This builder detects that and prints [pending]. Run the Colab notebook, copy `results/` back, then run `python scripts/build_viva_pdfs.py` again.
- README.md says "see 'Getting it onto Colab' below", but that section does not exist in the README.
- Training on the command line (`python -m src.train`) uses batch size 32 and lr 1e-3 for BOTH models by default. The notebook uses batch 64, lr 0.001 (CNN) and 0.0005 (EfficientNet). Always quote the notebook values; those produced the results.
- `build_efficientnet(freeze_backbone=True)` exists, but `get_model` never passes it, so feature extraction was never run. We have no numbers for it; only describe it as an option.
- The eval transform is `Resize((224, 224))`, which squashes non-square photos. PlantVillage photos are square (256×256), but PlantDoc photos are not (e.g. 512×384), so they get distorted. This is part of the measured domain gap.
- Leakage: our train and val share 0 leaf groups, but train and test share 2 leaf groups. These come from the authors' official split, which we use as is. Also, photos missing from the leaf map count as their own group, so the check cannot be 100% complete.
- The rejection threshold is chosen on the in-domain (lab) validation set. Softmax confidence is not calibrated, so field photos can be confidently wrong. The threshold is computed only for the two EfficientNet models. app.py falls back to 0.5 if `results/rejection/<name>.json` is missing.
- `RandomRotation(30)` fills the corners with black, and `RandomResizedCrop(scale 0.5-1)` can crop away the diseased spot, so some augmented images may no longer show the disease.
- PlantDoc's spider-mites folder has only 2 images, and several PlantDoc classes are small, so per-class field numbers are noisy. The PlantVillage test set has at most 60 images per class (Potato healthy 32, Apple cedar rust 52).
- Each experiment is run once with seed 42. Differences of 1-2 points between models may be noise. We report no confidence intervals or repeated runs.

- How PlantVillage was collected comes from the dataset paper (Hughes & Salathé 2015; Mohanty et al. 2016), not from our code. Check the wording against the paper before quoting details.
- The cosine LR schedule is stepped once per epoch with T_max = number of epochs and no warm-up, so the last EfficientNet epoch uses lr 3.35e-05 (see the LR table).
- The "~40 min on a T4" estimate is not measured yet. Read `train_time_sec` in `results/history_*.json` after the run.

25 likely viva questions for you (with model answers)

Basic

Q1. What is transfer learning?

Reusing a network trained on one task (ImageNet: 1.28M photos, 1000 classes) as the starting point for another task (our 38 leaf classes). We keep its learned filters, replace the last layer, and train further on our data.

Q2. Which pretrained model did you use, and why?

EfficientNet-B0 from torchvision with ImageNet weights (IMAGENET1K_V1). It is small (~4.06M parameters with our head) and fast on a free T4, yet accurate, and it gives a good Grad-CAM layer (1280×7×7).

Q3. What exactly did you change in the pretrained model?

Only the last layer: classifier[1] = nn.Linear(1280, 38) instead of 1280→1000. The Dropout(0.2) before it stays. See build_efficientnet in models.py.

Q4. Did you freeze any layers?

No. All layers are fine-tuned (freeze_backbone defaults to False and get_model never sets it), so all 4,056,226 parameters are trainable.

Q5. What is data augmentation?

Random changes to training images that keep the label: flips, crops, rotations, colour changes, blur. They are applied again every epoch, so the model sees more variety. Val/test images are never augmented.

Q6. What does 'strong' augmentation contain in your code?

RandomResizedCrop(224, scale 0.5-1.0), horizontal and vertical flips, RandomRotation(30), ColorJitter (brightness, contrast and saturation 0.4, hue 0.05), and GaussianBlur (kernel 9, sigma 0.1-3) with p=0.3.

Q7. What is domain shift?

The test data comes from a different distribution than the training data. Ours: lab photos (one leaf, plain background) vs field photos (clutter, several leaves, different light).

Q8. What is the confidence used for rejection?

The softmax probability of the predicted class (the largest of the 38 probabilities).

Intermediate

Q9. Why does transfer learning work for leaves when ImageNet has hardly any diseased leaves?

Early layers learn general things (edges, colours, textures) that are useful for any photo. Later layers combine them. Fine-tuning re-uses this and only adapts it to leaves, instead of learning it from 7,430 images.

Q10. Fine-tuning vs feature extraction: what's the difference?

Feature extraction freezes the pretrained layers and trains only the new head (48,678 parameters). Fine-tuning also updates the pretrained layers, with a small learning rate. We fine-tune; feature extraction is an option in the code that we did not run.

Q11. Why is the EfficientNet learning rate (0.0005) smaller than the CNN's (0.001)?

The pretrained weights are already good. Big steps would destroy them ('catastrophic forgetting'). The scratch CNN starts from random weights and can take bigger steps.

Q12. Explain your learning-rate schedule.

CosineAnnealingLR with T_max = number of epochs, stepped once per epoch: the LR starts at 0.0005 and follows half a cosine wave down towards 0. Big steps first, fine adjustments at the end. No warm-up.

Q13. Why only 6 epochs for EfficientNet?

A pretrained model converges very fast on PlantVillage, and the total budget had to stay within ~40 minutes on a free T4. We keep the best validation checkpoint anyway.

Q14. How do you measure the lab-to-field gap fairly?

Both accuracies are on the same 28 shared classes: PlantVillage test restricted to those classes vs PlantDoc. Domain gap = PV accuracy (28) − PlantDoc accuracy (28). The model still chooses among all 38 classes.

Q15. Why should colour jitter and blur help on field images?

Field photos have different light, white balance and focus. If the training images vary in brightness, colour and sharpness, the model cannot rely on the exact lab colours and has to use more robust features.

Q16. How is the rejection threshold chosen?

On the validation set only: the 5th percentile of the confidence of correct validation predictions (keep_correct=0.95). So about 95% of correct lab answers are still accepted. The test sets are never used to choose it.

Q17. What are coverage and 'accuracy on answered'?

Coverage = the share of images whose confidence is above the threshold (the model answers). Accuracy on answered = accuracy on those images only. Raising the threshold lowers coverage and usually raises accuracy on answered.

Tricky

Q18. Why EfficientNet-B0 and not ResNet50?

Both are valid. B0 has ~4M parameters vs ~25M for ResNet50, so it trains faster on a free T4 and runs fast in the Gradio app, with similar ImageNet accuracy. With more time we could compare both.

Q19. If strong augmentation lowers PlantVillage accuracy a bit but raises PlantDoc accuracy, is that bad?

No, that is the expected trade-off. The lab test is easy and over-optimistic. Field accuracy is what matters in practice, and a smaller domain gap means more robust features. (Check the real numbers in the table.)

Q20. Can augmentation alone close the domain gap?

Probably not fully. It only imitates some differences (zoom, angle, light, blur). It cannot add cluttered backgrounds, several leaves or new disease appearances. Real field training data or domain adaptation would be the next steps.

Q21. Why not choose the rejection threshold on PlantDoc, since that is where it matters?

Then PlantDoc would no longer be an unseen test set, and the reported numbers would be optimistically biased. In a real project you would use a separate field validation set.

Q22. The model is 95% confident on a field photo but wrong. How is that possible?

Softmax confidence is not calibrated, especially for inputs unlike the training data. Networks are often over-confident out of domain. Temperature scaling or out-of-distribution detection could help; we did not do that.

Q23. What would you do with more time to improve field accuracy?

Add some labelled field images to training (or fine-tune on part of PlantDoc with a held-out test part), random background augmentation for EfficientNet, longer training, test-time augmentation, model calibration, and several seeds to report variance.

Q24. Does RandomResizedCrop risk producing wrong labels?

Yes, a little. With scale 0.5 the crop can miss the lesion, so a 'diseased' image may show only healthy tissue. Scale 0.5-1.0 keeps at least half of the image area to limit this.

Q25. Is fine-tuning all layers always better than freezing?

No. With very little data, freezing avoids overfitting and is faster. We have 7,430 training images and a domain quite different from ImageNet, so fine-tuning everything with a small LR is a reasonable choice, but we did not run the frozen variant to prove it.